

PEERS Deployment Guide

Part 1: Preparing a model for deployment

Your model should be uploaded to Huggingface. Before it can be deployed to Huggingface, it needs a handler.py.

1. First you will need to install Git LFS.

Follow the instructions here to do so:

<https://docs.github.com/en/repositories/working-with-files/managing-large-files/installing-git-large-file-storage>

2. Check out your model from Huggingface to your local environment using git:

```
git clone https://huggingface.co/peers-ai/<model_name>
```

3. Now add a handler.py to the local directory.

Here is a below sample:

<<<

```
# handler.py — Fixed for Qwen3-30B-A3B LoRA (no hangs, 4-bit, direct generate)
```

```
import json
import logging
from typing import Any, Dict, List, Optional
```

```
import torch
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    StoppingCriteria,
    StoppingCriteriaList,
    BitsAndBytesConfig,
)
```

```
logging.basicConfig(level=logging.INFO)
log = logging.getLogger(__name__)
```

```
class StopOnStringsLoose(StoppingCriteria):
```

```

def __init__(self, tokenizer, stop_strings: List[str]):
    self.stop_ids = [tokenizer.encode(s, add_special_tokens=False) for s in (stop_strings or [])
if s]

```

```

def __call__(self, input_ids: torch.LongTensor, scores: torch.FloatTensor, **kwargs) -> bool:
    seq = input_ids[0].tolist()
    for ids in self.stop_ids:
        L = len(ids)
        if L and len(seq) >= L and seq[-L:] == ids:
            return True
    return False

```

```

class EndpointHandler:

```

```

    def __init__(self, model_dir: str):
        log.info("Loading tokenizer/model (4-bit)...")
        torch_dtype = torch.bfloat16
        quant_config = BitsAndBytesConfig(
            load_in_4bit=True,
            bnb_4bit_quant_type="nf4",
            bnb_4bit_compute_dtype=torch_dtype,
            bnb_4bit_use_double_quant=True,
        )

```

```

        self.tokenizer = AutoTokenizer.from_pretrained(model_dir, trust_remote_code=True)
        self.model = AutoModelForCausalLM.from_pretrained(
            model_dir,
            torch_dtype=torch_dtype,
            device_map="auto",
            quantization_config=quant_config,
            trust_remote_code=True,
        )

```

```

        # Fix EOS/PAD

```

```

        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token
        self.model.config.pad_token_id = self.tokenizer.pad_token_id
        self.model.config.eos_token_id = self.tokenizer.eos_token_id

```

```

        log.info("Model ready.")

```

```

    def _build_prompt(self, data: Dict[str, Any]):
        messages = data.get("messages") or [{"role": "user", "content": data.get("inputs", "")}]
        # Esoteric instructions + /no_think
        messages.insert(0, {"role": "system", "content": "Be concise. Answer in 1–2 paragraphs. No meta-text. /no_think"})

```

```

    return self.tokenizer.apply_chat_template(messages, tokenize=False,
add_generation_prompt=True)

def __call__(self, data: Dict[str, Any]) -> Dict[str, Any]:
    params = data.get("parameters") or {}
    max_new_tokens = int(params.get("max_new_tokens", 256))
    temperature = float(params.get("temperature", 0.2)) # Low for esoteric focus
    top_p = float(params.get("top_p", 0.6))
    repetition_penalty = float(params.get("repetition_penalty", 1.5)) # High to avoid loops
    stop_list = params.get("stop", []) or []
    if isinstance(stop_list, str): stop_list = [stop_list]

    prompt = self._build_prompt(data)
    inputs = self.tokenizer(prompt, return_tensors="pt").to(self.model.device)

    stopper = StoppingCriteriaList([StopOnStringsLoose(self.tokenizer, stop_list)])

    log.info("Starting generation...")
    with torch.no_grad():
        try:
            outputs = self.model.generate(
                **inputs,
                max_new_tokens=max_new_tokens,
                do_sample=True,
                temperature=temperature,
                top_p=top_p,
                repetition_penalty=repetition_penalty,
                eos_token_id=self.tokenizer.eos_token_id,
                pad_token_id=self.tokenizer.pad_token_id,
                stopping_criteria=stopper,
            )
            text = self.tokenizer.decode(outputs[0][inputs["input_ids"].shape[-1]:],
skip_special_tokens=True)
        except Exception as e:
            log.error(f"Generation failed: {e}")
            text = "[Error: Generation failed]"
    log.info("Generation complete.")

    # Post-trim stops
    for s in stop_list:
        i = text.find(s)
        if i != -1:
            text = text[:i].strip()

    torch.cuda.empty_cache()

```

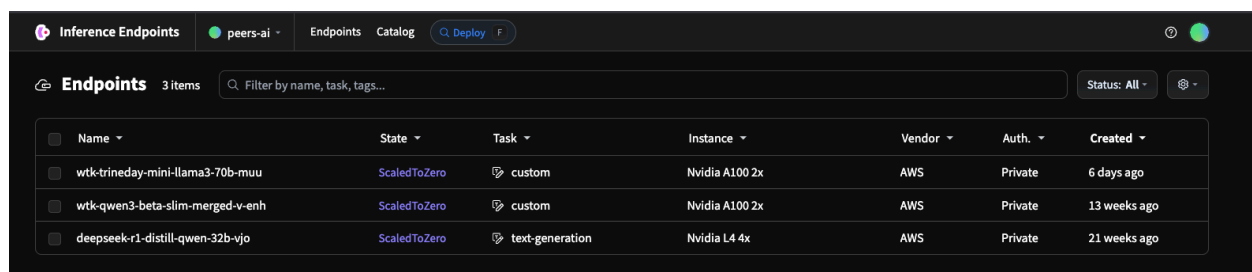
```
return {"generated_text": text}
```

>>>

4. Commit your model back to Huggingface.

Part 2: Deploying on Huggingface

We deploy our models on Huggingface Endpoints. This guide will explain how to deploy a peer-ai model on HF Endpoints.



The screenshot shows the Huggingface Inference Endpoints interface. At the top, there are tabs for 'Inference Endpoints', 'peers-ai', 'Endpoints', and 'Catalog'. A 'Deploy' button is visible. Below the tabs, there's a search bar and a 'Status: All' filter. The main content is a table with columns: Name, State, Task, Instance, Vendor, Auth., and Created. Three models are listed:

Name	State	Task	Instance	Vendor	Auth.	Created
wtk-trinaday-mini-llama3-70b-muu	ScaledToZero	custom	Nvidia A100 2x	AWS	Private	6 days ago
wtk-qwen3-beta-slim-merged-v-enh	ScaledToZero	custom	Nvidia A100 2x	AWS	Private	13 weeks ago
deepseek-r1-distill-qwen-32b-vjo	ScaledToZero	text-generation	Nvidia L4 4x	AWS	Private	21 weeks ago

Another good guide is this one:

<https://huggingface.co/blog/inference-endpoints-llm>

The path to deployment is different whether you are using a public model or a private model on your account.

If you are deploying a private model, then you have to do so from within your Huggingface account page and not from Huggingface Endpoints.

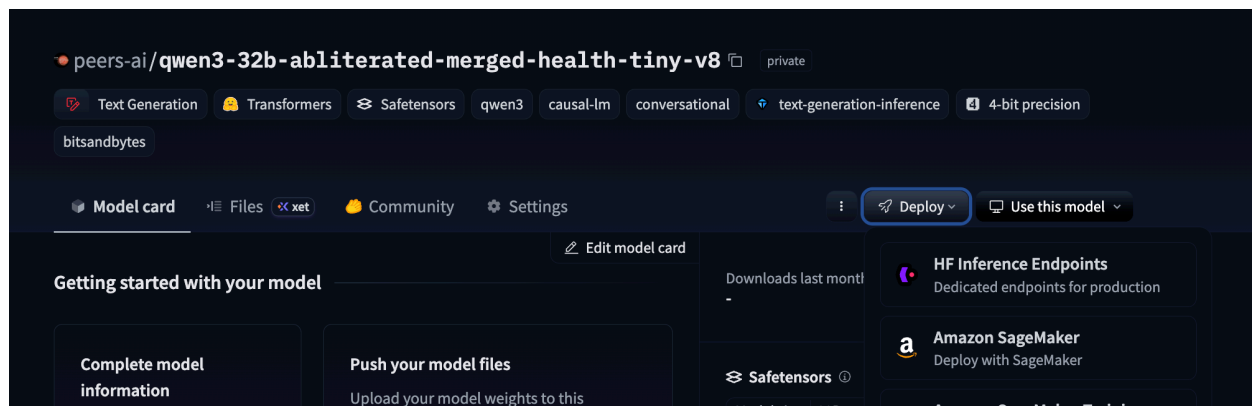
Peers-ai huggingface account page:

<https://huggingface.co/peers-ai/models>

Since we assume we are dealing with our peers-ai models, we will show instructions how to deploy a private model

3. From the peers-ai account page, choose the model you want to deploy and go to its page.

4. Find the Deploy button on the model's page and then choose "HF Inference Endpoints." The Deploy button should be on the middle-right. When you click, a sub-menu of options will appear. Click the option "HF Inference Endpoints."



Note: there are cases when the “Deploy” button doesn’t appear at all. If this is the case, follow the below instructions.

This is a known issue and somewhat confusing. But the Deploy button will only appear for transformer models. See the link here for more info.

<https://discuss.huggingface.co/t/deploy-button-not-showing-fine-tuned-llama-3-1/103630>

You will need to add a README.md file to your model files and then add the following to the top:

```
---
library_name: transformers
tags:
  - text-generation
  - causal-lm
---
```

Add this change and commit it. Then you should see the Deploy button show up.

Part 3: Configuring the new instance on HF Endpoints

The Create Endpoint page will open up.

Regarding configuration:

- Under “Hardware Configuration” we have been using AWS by default though you can select whichever you feel. Then you will want to select GPU. Then a reasonable hardware default should be selected for you.
- Authentication should be “Private.”
- For Autoscaling, we can use min 0 and max 1. Autoscale to zero after either 30 min or 1 hour with no activity.
- Everything else can be kept at default.

Once you are ready, then click “Create Endpoint”.

← Back to Catalog

Create Endpoint

peers-ai

qwen3-32b-abliterated-merged-health-tiny-v8

⚠ This model is not part of our Model Catalog, and does not come with verified configuration. It might behave unexpectedly.

peers-ai /

qwen3-32b-abliterated-merged-sis

\$1.80 / h

per running replica

cURL

Create Endpoint

Hardware Configuration

Contact us if you'd like to request a custom solution or instance type.

aws Amazon Web Services

Microsoft Azure

Google Cloud Platform

CPU GPU INF2

Nvidia L40S

1 GPU

Nvidia L40S

1x GPU · 48 GB

7x vCPUs · 30 GB

\$1.8 / h

🇺🇸 N. Virginia

us-east-1

Suggested

Best performance/price ratio for selected model and accelerator.

Authentication

Private

Public

Authenticated

Only you can access your endpoint, using a Hugging Face Token generated from your personal account.

Autoscaling

0 to 1 Replica / Scale-to-zero after 60 min

Inference Engine

Default

Container Configuration

No arguments or commands defined

Environment Variables

No env variables defined

Endpoint Tags

No tags defined

Network

AWS Private Link

Part 4: Managing the instance

Now the endpoint will be created. The dashboard will look like this.

wtk-qwen3-beta-slim-merged-v-enh

Scaled to Zero

Pause

Wake up

Overview

Playground

Analytics

Logs

Settings

Scaled to Zero due to Inactivity

Use the button above or send a request to wake your Endpoint.

Endpoint URL

Help

https://cr41uamktrsydg3d.us-east-1.aws.endpoints.huggingface.cloud

Configuration

Private

custom

Created Nov 17, 2025 by peers-ai · Last Edited Feb 23 at 8:45 PM by platform

Model

Up-to-date

peers-ai/wtk-qwen3-beta-slim-merged-v4-A

Revision fb3640c

Default Engine

No versioning available for Default Engine

Instance

\$ 5 / h while running

aws

us-east-1

GPU · Nvidia A100 · 2x GPUs · 160 GB

Scale-to-zero After 1 hour with no activity

VPC Service Name

This endpoint doesn't have a VPC config set

Usage

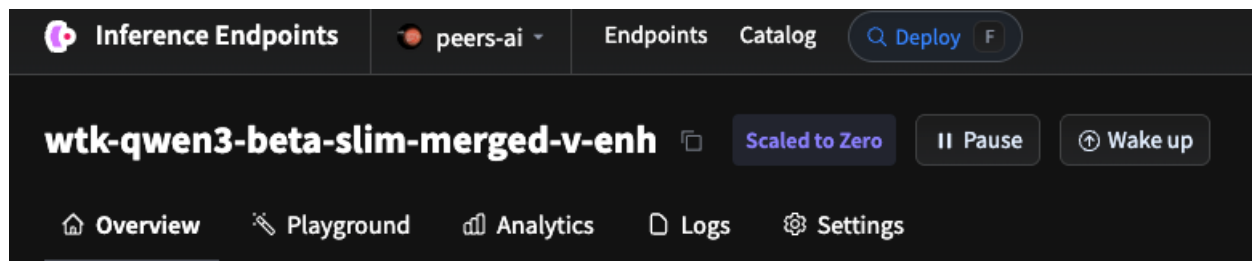
Past Invoices

Instance Type	Compute Time	Rate (USD/hour)	Cost (USD)
2x Nvidia A100	779 minutes	5.00	64.92
Total			\$ 64.92

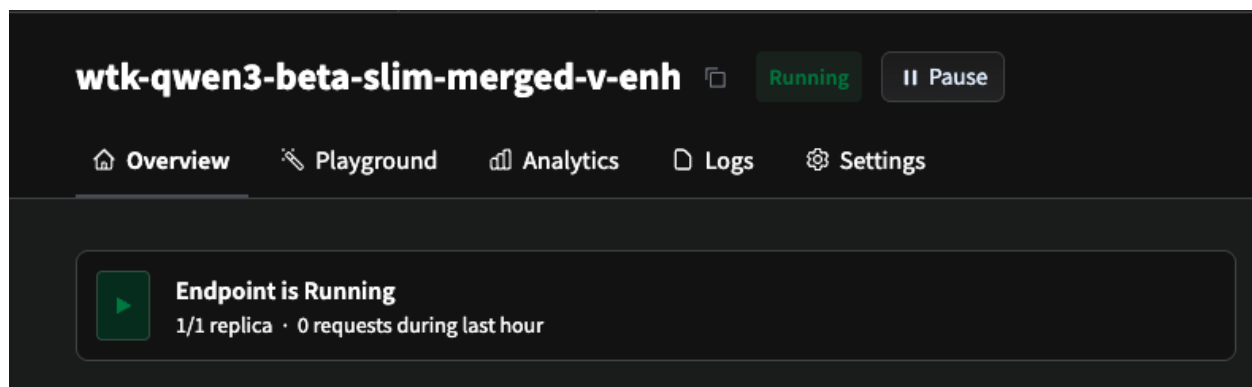
There are a number of important operations which you will commonly do.

How to wake up an instance

When the instance is asleep, click the “Wake up” button to start it.



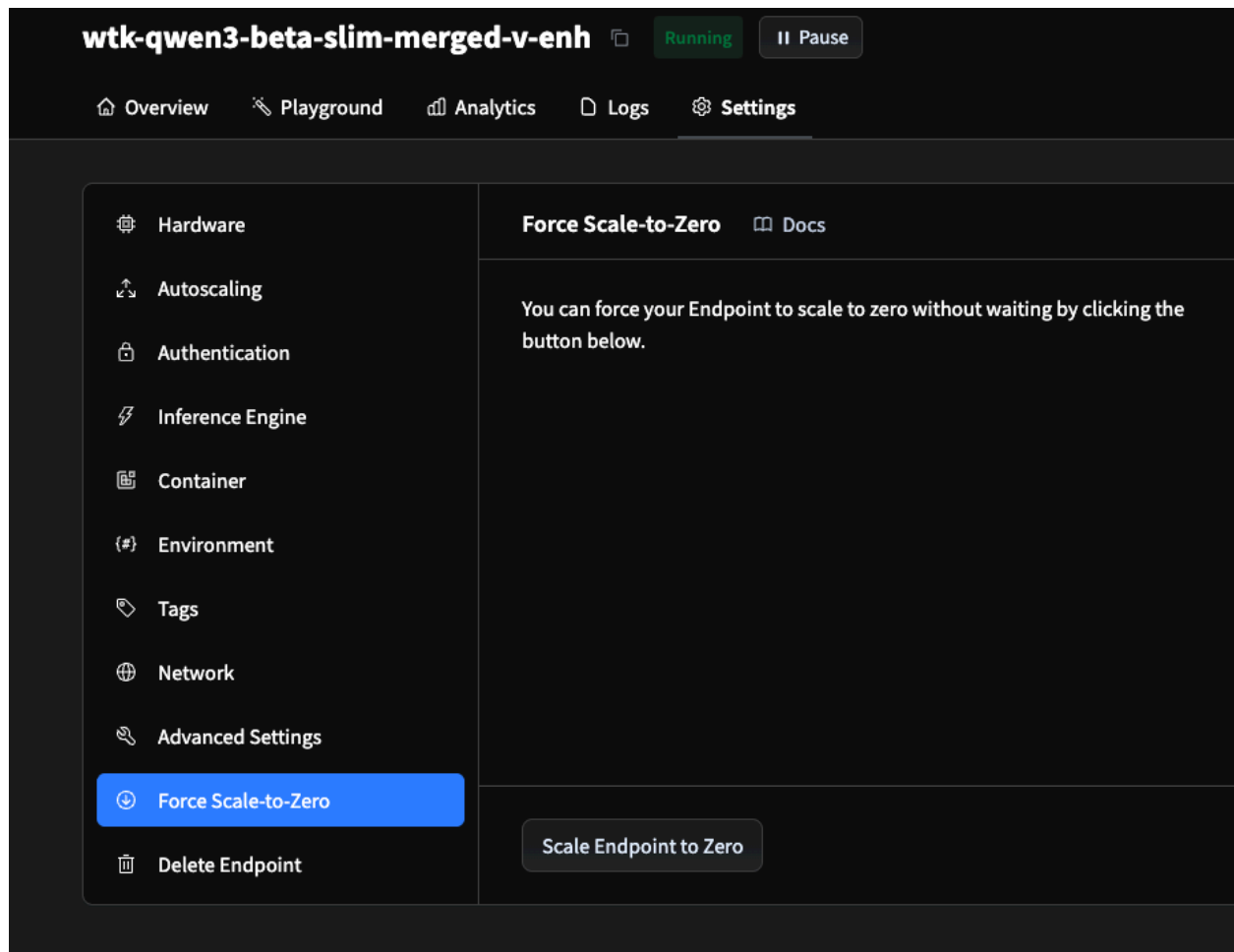
The instance status will say “Initializing” when it is starting up. It will take a few minutes to start up. When ready it will look like the below.



How to shutdown a running instance

To shutdown an instance when it is running, click Settings entry on the menu. The settings menu should appear.

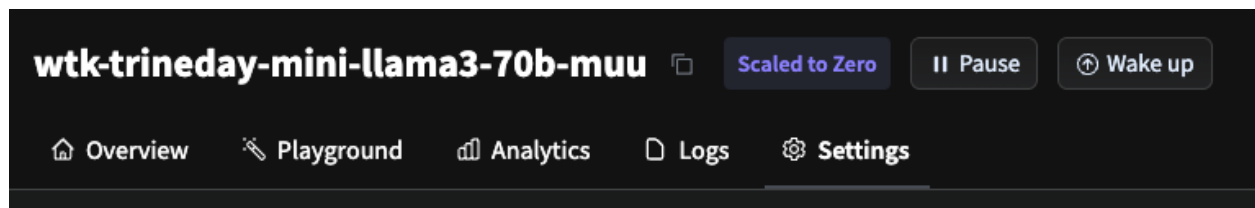
Then you want to find the “Force Scale-to-Zero” button. Then you will get the submenu.



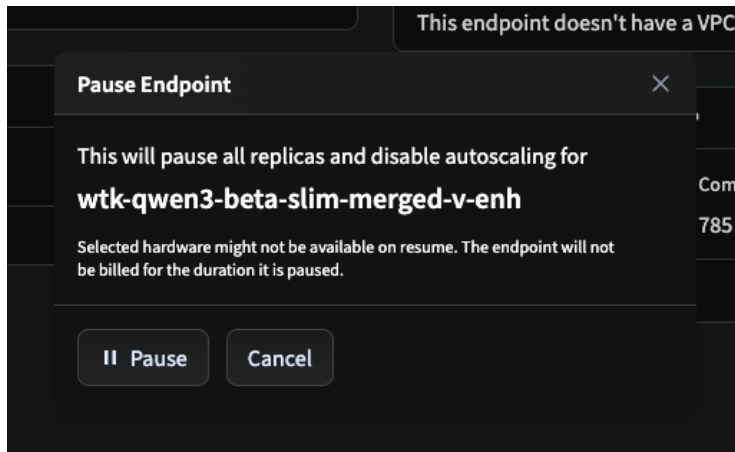
Click the “Scale Endpoints to Zero” button. The status should change to “Scaled to Zero.”

How to pause an instance

Pausing an instance stops the instance and makes it such that incoming queries do not wake up the instance.



Click the Pause button and then another pop-up comes up.



Click the Pause button on the pop-up.

The endpoint will now be paused and will not wake up on request until you press the Resume button.

